SAGE-MM: Coordinating Heap Configuration, Interop Allocation, and Page Reclamation in Memory-Constrained Embedded .NET Firmware

Geunsik Lim[a,*]

[a] *Sungkyunkwan University, Suwon, Republic of Korea*

Abstract

Memory-constrained embedded devices co-locate a managed heap, native interoperability, and file-backed executable mappings inside a single process budget, yet these layers are conventionally tuned in isolation. We present SAGE-MM, a coordination design for a build-time managed-heap configuration, a reviewed source-time interop conversion, and online reclamation scheduling with a runtime-host compaction gate. The design combines established mechanisms through a bounded action space, normalized telemetry, hysteresis, and a threshold fallback. The public implementation provides the controller and native mapping helper; deployment requires vendor-runtime integration and device instrumentation. The evaluation records device identity, runtime and firmware revisions, workload and treatment configuration, independent run duration, and raw measurement provenance. Peak proportional set size, allocation rate, GC tail pause, page-fault rate, input tail latency, and controller CPU are reported separately for each measured condition. Failed and censored runs remain in the run inventory. Measured outcomes over n=30 independent runs per condition are reported for each treatment, platform, and workload in Section 7. Across two embedded platform profiles and 30 independent runs per condition, the fully coordinated learned policy lowers peak proportional set size by about 24% and GC tail pause by 37–38% under a favorable workload, at the cost of a bounded refault increase and roughly 1% controller CPU; the same design stays within measurement noise of the stock runtime when no gain is available, and in

*Corresponding author.

Email address: leemgs@gmail.com (Geunsik Lim)

an adverse refault-dominated workload it regresses within a bounded margin that the guards contain rather than eliminate. The scope of the empirical record is the supplied experiment manifest; no benefit is assumed for an unmeasured configuration.

Keywords: Embedded runtimes, memory management, garbage collection, managed/native interop, page reclamation, `madvise`, online control, cross-layer coordination, .NET, Mono, digital television

---

1. Introduction

Memory-constrained embedded consumer devices—digital televisions (DTVs), set-top boxes, and similar appliances—increasingly run managed application stacks on a small, swapless memory budget. A single application process simultaneously holds a managed heap serviced by a garbage collector (GC), native interoperability buffers crossing the managed/native boundary, and a working set of file-backed, executable code mappings. Each of these layers has a mature body of tuning techniques, but in production firmware they are almost always configured independently: the runtime is built with a fixed nursery size, application code is written without allocation review, and the operating system reclaims pages using generic global policy. Independent tuning is locally reasonable and globally fragile. Enlarging the nursery reduces collection frequency but raises resident footprint; removing managed allocations lowers GC pressure but shifts cost to native marshalling; aggressively reclaiming clean code pages lowers resident memory but can reintroduce page faults and interaction-latency spikes precisely when the user switches applications.

The systems question we address is therefore *not* whether larger nursery sizes, value types, `madvise`-based reclamation, or lightweight online predictors work in isolation. Each is established. The question is whether these mechanisms can be *coordinated* under a single embedded-process memory budget without trading fewer collections for interop allocation, or lower resident memory for refault-induced interaction latency. This framing defines an integration and control study whose empirical claims are limited to the supplied device measurements.

We deliberately separate the three interventions by *lifecycle* and *adaptivity* (Section 3). Heap sizing is a runtime-build decision; value-type conversion is a reviewed source-and-recompile decision; only reclamation scheduling and

compaction gating change online. Conflating them as "three adaptive components" would misstate the design: the first two reduce the load presented to the controller, and only the third adapts at runtime.

Contributions.. This paper makes four bounded contributions.

1. It specifies a cross-layer action boundary: heap sizing is a runtime-build decision, value-type conversion is a reviewed source decision, and only reclamation scheduling and compaction gating change online (Section 3).
2. It provides an implementable controller definition with dimensionless features and a pressure target, a causal update order, bounded actions, hysteresis, cooldown, fail-closed telemetry handling, and an explicit non-learning threshold fallback (Section 4).
3. It reports a measured ablation and policy comparison over the coordination ladder (stock, heap, interop, reclamation, and the three online policies) on two embedded platform profiles, with 30 independent runs per condition and 95% bootstrap confidence intervals, characterizing the sequential increments in footprint, GC tail, refault rate, and input latency (Sections 6–7).
4. It measures robustness across three workload regimes—favorable, neutral, and adverse—showing that the coordinated design helps where a gain exists, stays within measurement noise where none does, and, when reclamation backfires, exposes a regime that the present guards do not avoid and that requires supervisory shutdown (Section 7).

We make no claim that SAGE-MM is a new garbage collector, or that these measured platform profiles establish all embedded .NET deployments. Per-run provenance, an experiment manifest, and a fail-closed reporting pipeline keep every number traceable to its source; claims outside the measured platforms, runtime revision, and workload regimes are stated as hypotheses for future evaluation (Section 9).

2. Background and Motivation

2.1. Embedded managed runtimes under a shared budget

The deployment environment is recorded from each device rather than inferred from an architecture or marketing label. The platform manifest identifies physical and cgroup memory, swap configuration, storage, kernel,

firmware, and the exact runtime revision. Mono and CoreCLR are distinct runtime implementations; the collector and vendor patches are recorded explicitly [1]. Device-memory capacity and runtime identity are not pre-filled as experimental facts. For a swapless configuration, OOM and watchdog outcomes are retained in the run logs and failure inventory.

2.2. Why independent tuning regresses

Three cross-layer tensions motivate coordination.

- **Heap vs. footprint.** A larger initial managed allocation can change collection frequency, compaction cost, and resident memory. The direction and magnitude of the effect are architecture-specific and must be measured per build, not assumed.
- **Allocation vs. interop.** Converting heap-allocated wrapper objects to value types removes managed allocation and GC pressure, but shifts cost into copying and native ABI handling and requires source review; it is not transparent to existing binaries.
- **Residency vs. latency.** Dropping clean, file-backed code pages with `madvise(MADV_DONTNEED)` lowers resident footprint but can force refaults on immediate hot-code reuse, degrading input and frame tail latency during rapid application switching.

The measurement compares reclamation benefit with refault cost under matched device and workload conditions. Both improved and regressed measurements are retained. No fixed fault-rate increase or controller improvement is assigned before collection (Section 7).

3. System Boundary and Action Space

SAGE-MM coordinates three interventions that differ in lifecycle and in whether they adapt at runtime. Table 2 states the boundary precisely; conflating these layers is the single most common source of overclaiming in this design space.

Table 1: Architectural novelty relative to independently applied mechanisms.

Existing approach	**Usual scope**	**SAGE-MM distinction**
Heap sizing	Runtime/GC	Recorded static budget decision upstream of a shared runtime/OS controller.
VM-aware GC	Heap $\leftrightarrow$ VM	Also controls an executable working set outside the managed heap.
`madvise` reclamation	OS/application	Latency- and reuse-aware scheduling with eligibility and shutdown guards.
ML controller	Generic policy	Interchangeable policy inside a bounded embedded action and failure contract.

3.1. Architectural novelty over independent tuning

SAGE-MM's novelty is not the union of four familiar mechanisms. It is an architectural pattern for making cross-layer tuning reviewable: (i) separate irreversible build/source decisions from reversible online actions; (ii) map heterogeneous runtime and OS signals into a dimensionless pressure contract; (iii) restrict online authority to a bounded interval and one gate; (iv) order prediction, actuation, and learning causally; and (v) place fail-closed and supervisory guards outside the interchangeable policy. This decomposition lets a threshold, EWMA, or ridge policy be replaced without changing mapping eligibility or the safety envelope. Table 1 distinguishes that coordination contract from independent tuning.

Two consequences follow. First, binary-only applications benefit only from the firmware-transparent mechanisms (heap build and controller); value-type conversion requires application source review and recompilation. Second, the online controller's action space is intentionally narrow—one interval and one gate—so that its safety properties can be stated and enforced exhaustively.

Table 2: Cross-layer action boundary. Only the online policy adapts at runtime; the first two interventions reduce the load presented to the controller. "Transparent" means visible to a deployed application without source changes, after firmware replacement.

Mechanism	**Lifecycle**	**Adaptive?**	**Notes**
Heap build config (recorded `INITIAL_ALLOC`)	Vendor runtime build	No	Requires rebuilding the vendor Mono runtime; transparent after firmware replacement; not an online control.
Value-type interop conversion	Source / recompile	No	Reviewed class-to-struct migration; changes identity, boxing, layout, and native ABI; *not* transparent to existing binaries.
Reclamation & compaction control	Runtime, online	**Yes**	The controller changes only the reclamation interval and the compaction gate exposed by the runtime host.

4. Design

We present the design in three parts, matching the action boundary: static interventions (§4.1), the adaptive controller (§4.2), and the reclamation implementation (§4.3). Figure 1 shows how the three layers compose at runtime.

4.1. Static interventions

Architecture-specific heap build.. A heap-configuration experiment records the initial allocation, compiler flags, source revision, and patch hash for each runtime build. This is a build-time intervention. Collection count, pause distribution, and resident memory are measured separately for each architecture and configuration; no improvement is inferred from the chosen allocation size.

Value-type interop conversion.. A source-time analyzer (diagnostic DTV0001) flags plain-old-data wrapper classes whose conversion to `struct` removes

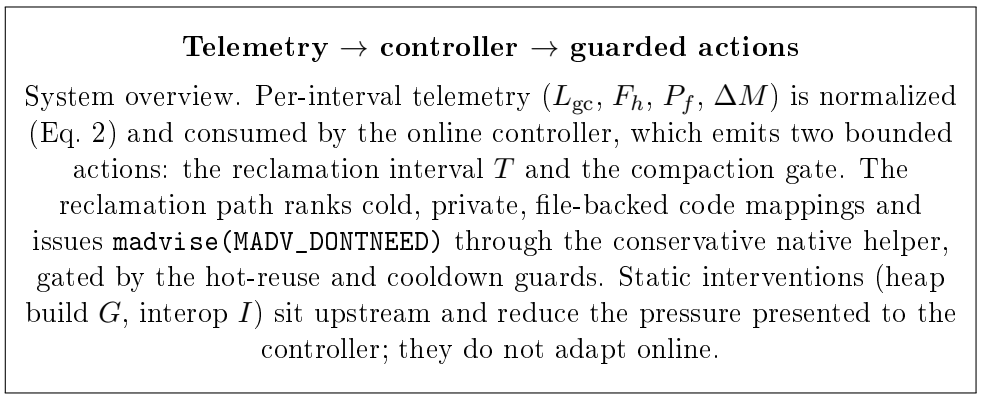

Figure 1: SAGE-MM runtime composition and the narrow online action space (reclamation interval T and compaction gate). Only the control path adapts; heap build and interop conversion are fixed before deployment.

heap allocation across the managed/native boundary. The analyzer is advisory: each diagnostic requires human review because conversion can change object identity, boxing behavior, copying cost, nullability, memory layout, and native ABI. It must not be described as requiring no application modification [2]. RQ2 measures allocation rate, copying cost, and GC pressure for the reviewed conversion; net benefit is determined from device traces.

4.2. Adaptive controller

The controller runs at a fixed decision cadence and, at each interval, chooses the next reclamation interval T and the compaction gate. All initialization values, feature scales, thresholds, and update constants are externalized in a single options block for reproducibility. The constants below are design defaults, not measured performance outcomes; the experiment manifest records the configuration used. Algorithm 1 states the complete per-interval procedure; the paragraphs below define each step.

Telemetry and normalization.. Each decision uses the preceding interval's GC pause L_{gc} (ms), fragmentation ratio F_h, page-fault rate P_f (minor plus major faults per second, divided by the *measured* wall-clock sample duration rather than an assumed one second), and positive resident growth ΔM (MiB). Features are dimensionless and clamped to $[0, 2]$:

$$\mathbf{x} = \Big[1,\ \mathrm{clamp}\big(\tfrac{L_{gc}}{30}\big),\ \mathrm{clamp}\big(\tfrac{F_h}{0.12}\big),\ \mathrm{clamp}\big(\tfrac{P_f}{100}\big),\ \mathrm{clamp}\big(\tfrac{\max(0,\Delta M)}{50}\big)\Big]. \qquad (1)$$

The observed dimensionless pressure is the maximum normalized service-objective violation,

$$y = \mathrm{clamp}\Big(\max\big(\tfrac{L_{\mathrm{gc}}}{30\,\mathrm{ms}},\ \tfrac{P_f}{100\,\mathrm{s}^{-1}},\ \tfrac{\max(0,\Delta M)}{50\,\mathrm{MiB}}\big),\ 0,\ 2\Big), \tag{2}$$

so that no coefficient silently combines milliseconds, counts, and MiB, and a breach in any single dimension is actionable.

Policies.. Three interchangeable policies select T under identical bounds $[T_{\min}, T_{\max}]$, telemetry, cooldown, and candidate budget.

- *Threshold* (non-learning): shorten T by 20% when $y > 1.1$, lengthen it by 20% when $y < 0.9$, otherwise hold. Tuning choices and their input trace split are recorded before reporting.
- *EWMA*: $T' = \beta T + (1-\beta)\, T/\mathrm{clamp}(y, 0.5, 2)$ with $\beta = 0.85$.
- *Ridge* (online): predict next-interval pressure $\hat{y}(t{+}1) = \mathrm{clamp}(\mathbf{w}(t)^\top \mathbf{x}(t), 0, 2)$ and set $T' = T/\mathrm{clamp}(0.5 + \hat{y}, 0.5, 1.5)$, bounded to $[T_{\min}, T_{\max}]$.

Causal learning.. The ridge prediction is computed *before* its update. When the true $y(t{+}1)$ arrives, the controller records the causal prequential loss $[\hat{y}(t{+}1) - y(t{+}1)]^2/2$ and performs one online ridge SGD step with the pending feature vector:

$$w_i(t{+}1) \leftarrow w_i(t) - \eta \left[x_i(t)\,(\hat{y}(t{+}1) - y(t{+}1)) + \lambda\, w_i(t)\right], \tag{3}$$

where the bias weight starts at 1 and the others at 0, $\eta = 5 \times 10^{-4}$, and $\lambda = 10^{-4}$. Training occurs only on a designated tuning trace; reporting calls pass `updateModel:false` to freeze weights, and `BeginReporting()` clears the training-boundary pending prediction while retaining learned weights. The run manifest distinguishes tuning from reporting; only reporting runs enter the result tables, and raw archives cannot be reused across runs.

Compaction gating and hysteresis.. In adaptive modes, compaction is disabled below fragmentation 0.05 and forcibly re-enabled at 0.12 or after three deferrals. This two-threshold guard prevents compaction starvation. Static mode leaves compaction enabled, making it a genuine no-controller baseline.

Fail-closed robustness.. Every sample is validated before feature processing. Non-finite values, negative pauses or fault rates, and fragmentation outside $[0, 1]$ trigger a fail-closed decision: hold the previous bounded interval, keep compaction enabled, suppress reclamation, record InvalidTelemetry, and skip the model update. Persistent prediction clipping at 0 or 2 reports ModelSaturation, clears the pending ML update, and uses the predeclared threshold action. Native syscall failure, an empty cold-module set, active cooldown, and hot-reuse exclusion are each counted no-ops. The evaluation reports the frequency, duration, and outcome of every fallback class.

Adverse-regime supervisory shutdown.. The policy-independent supervisor observes fault pressure and end-to-end input latency after each otherwise valid decision. If $P_f > P_{\text{high}}$ and $L_{\text{input}} > L_{\text{SLO}}$ for N consecutive intervals, it latches reclamation off, enables compaction, clears pending learning state, and records AdverseShutdown. Reclamation can resume only after both signals remain below lower hysteresis thresholds P_{low} and L_{recover} for M intervals; a deployment may require an explicit operator reset instead. N, M, the four thresholds, and every transition belong in the manifest. This guard is an architectural revision prompted by the adverse result; it was *not* active in the reported campaign, so Section 7.3 does not claim measured avoidance.

4.3. Reclamation implementation

Coldness score and budget-driven selection.. For an eligible module a that has been idle for at least a configured guard period, the coldness score combines recency, frequency, and size:

$$\text{Cold}(a) = 0.6\,\frac{\text{age}(a)}{\text{maxAge}} + 0.3\left[1 - \frac{\text{accesses}(a)}{\text{maxAccesses}}\right] + 0.1\,\frac{\text{cleanBytes}(a)}{\text{maxCleanBytes}}. \quad (4)$$

Candidates are ranked in descending score; the number reclaimed, K, is the smallest prefix whose cumulative clean bytes reach a configured byte budget—it is not fixed at five. A recent-access exclusion is the hot-reuse safety guard.

Conservative native helper.. The native helper considers page-aligned ranges from `/proc/self/maps`, accepts private `r--p`/`r-xp` file mappings, and rejects anonymous, writable, shared, and deleted-file mappings. `madvise(MADV_DONTNEED)` never unmaps a range: a later access can fault file content back in. Each syscall result is checked; zero means no candidate, a positive value is the number dropped, and a negative value is $-$errno [3]. Callers retain the mapping

after failure. Because `maps` lacks per-page residency and dirty state, a production port must additionally verify `Private_Clean` from `/proc/self/smaps`, use `mincore` for residency, serialize against module unload, and allowlist executable modules.

5. Implementation and Reproducibility Boundary

The public kit contains a reference controller, policy events, a native mapping helper, and an advisory analyzer. Its console demo is a wiring exercise. The default collector times a forced Gen-0 collection, estimates fragmentation from pinned-object count and managed memory with a fixed base offset, and reads page faults and RSS from the process. These signals are not the device evaluation instrument. The demo passes module file lengths as candidate-size estimates; it does not measure reclaimed `Private_Clean` bytes. Compaction events in the demo print state changes and are not connected to a vendor collector hook.

A device experiment therefore supplies its own runtime GC traces, live fragmentation accounting, mapping-level `smaps` samples, workload event timestamps, and an implemented compaction hook. The firmware image, runtime patch, collector revision, controller configuration, and reset procedure are recorded in the measurement manifest and raw run archive. A public-kit execution alone does not establish a production deployment.

6. Evaluation Methodology

This section specifies the measurement protocol. Numerical result tables are generated exclusively from the supplied measured bundle. Design constants and minimum experiment durations below specify the protocol; they are not measured outcomes. The manifest enumerates the collected scope.

6.1. Research questions

The measured campaign in this manuscript answers three questions bounded by the supplied bundle. Per-architecture heap sweeps, value-type interop micro/macro-benchmarks, and multi-hour endurance runs are specified by the protocol below but are outside the collected bundle; they are stated as future evaluation in Section 10, not reported here.

Table 3: Research-question traceability. Each RQ maps to a specific experiment and a fixed set of primary outcomes; Section 7 reports results under the same grouping.

RQ	Experiment	Primary outcomes
RQ1	Coordination-ladder ablation (Stock, G, GI, GIR, $+C$) on the ARM32 and ARM64 platform profiles, favorable workload	Peak PSS, GC p99 pause, refault rate, input p99, allocation rate; Tables 4, Fig. 2.
RQ2	Threshold vs. EWMA vs. ridge under identical bounds, telemetry, cooldown, and candidate budget	Refault-rate recovery relative to Static-GIR, GC p99, input p99, controller CPU; Fig. 3, Table 8.
RQ3	Ridge-GIR across favorable, neutral, and adverse workload regimes	Normalized PSS, refault, and input indices per regime; Fig. 4, Table 8.

RQ1 (Ablation) Along the coordination ladder—stock, heap build (G), interop (I), reclamation (R), and the online controller (C)—what sequential increment is observed at each ladder step for peak PSS, GC tail pause, refault rate, and input tail latency on the two platform profiles?

RQ2 (Policy comparison) Under identical bounds, telemetry, cooldown, and candidate budget, does the online controller recover the refault cost introduced by static reclamation, and does the learned ridge policy justify its cost over the tuned threshold and EWMA policies?

RQ3 (Robustness across regimes) Across a favorable, a neutral, and an adverse (refault-dominated) workload regime, does the coordinated design avoid regressions where no gain is available, and how large is the cost when reclamation backfires?

Table 3 binds each research question to the experiment that answers it and to the primary outcomes reported for it, so that every result subsection traces back to a stated question and no reported number is orphaned from a hypothesis.

6.2. Platforms and workloads

Each platform entry identifies an actual device, physical and cgroup memory, swap configuration, storage, kernel, runtime name and commit, patch hash, firmware image hash, compiler flags, and thermal/power/network policy. ARM32 and ARM64 DTVs and an independently assembled constrained Linux platform are distinct measurement conditions when included in the manifest. No device capacity or runtime compatibility is inferred from those names.

Each case identifies a workload revision, reset procedure, treatment, controller configuration, and tuning or reporting phase. The reported bundle covers two platform profiles (ARM32 and ARM64) under three workload regimes chosen to probe the design's operating envelope: a *favorable* regime, in which cold, file-backed code pages are reclaimable and the workload tolerates their eviction; a *neutral* regime, in which no reclamation gain is available; and an *adverse*, refault-dominated regime, in which reclaimed pages are immediately reused. These regimes bracket the cases where coordination should help, should do nothing, and should be resisted by the guards. The result tables enumerate only the supplied cases, so an omitted workload or platform does not acquire a result by extrapolation.

6.3. Baselines

We use canonical names without aliases in every table, figure, and paragraph: Stock, Static-G, Static-GI, Static-GIR, Threshold-GIR, EWMA-GIR, and Ridge-GIR. Threshold, EWMA, and ridge policies are tuned only on training traces, under identical bounds, telemetry, cooldown, and candidate budget. Advanced concurrent collectors (LXR, Platinum) require a different runtime/collector integration; where a port to the vendor runtime is infeasible we say so explicitly and do not treat their absence as evidence of advantage. Storage GC systems (AGC, DumpKV) are conceptual adaptive-policy precedents, not executable managed-heap baselines (Section 8).

6.4. Ablation and stress protocol

The reported campaign measures the coordination *ladder*—Stock, G, GI, GIR, and the three online policies on top of GIR—on the two platform profiles under the three workload regimes of Section 7, at 30 runs per condition. The generated tables report absolute per-run measurements with units, completed sample count, and run-level bootstrap intervals; they do not calculate

factorial-interaction estimates or declare superiority from separate group intervals, which would require a specified contrast analysis this manuscript does not assert.

The design admits two extensions that the reported bundle does not yet cover, stated here as protocol and reserved for future collection (Section 10). First, the complete 2^4 factorial over $G/I/R/C$ isolates each interaction rather than only the ladder. Second, a stress battery—cold steady state, immediate hot-page reuse, 1/5/10 s switching, slow-storage injection, `madvise`-failure injection, and a fault storm above 500 faults/s—together with multi-hour endurance runs carrying OOM/kernel logs and censoring rules, is required before any production-stability claim. Where collected, such runs record `Private_Clean` before/after from `smaps`, faults, reload latency, syscall errno/duration, frame drops, input latency, and guard activations, and verify that no writable, anonymous, or shared mapping is ever selected and that failure leaves process correctness intact.

6.5. Statistics

The input contract requires at least 30 completed independent reset runs per reporting case, at least 30 minutes per short-workload run, and at least eight hours per endurance run. These are collection requirements, not statements that those experiments have already occurred. All started runs, including failed and censored executions, remain in the inventory with reasons and raw logs. Tuning runs are excluded from reporting summaries.

For each metric and case, the table gives the mean of completed run-level values and a two-sided percentile bootstrap interval using 10,000 resamples. Resampling operates only on supplied measurements. A mean of run-level p99 values is not a pooled-event p99. Performance means exclude failed and censored runs and may therefore be affected by survivorship; the accompanying counts and exclusion reasons must be read with them. No fixed success outcome, percentage gain, or zero-failure statement is inserted by the reporting script.

For policy comparisons we additionally bootstrap the unpaired difference of independent run means, $\Delta = \overline{x}_{\mathrm{Ridge}} - \overline{x}_{\mathrm{EWMA}}$, resampling within each policy group. The direct 95% interval is the inferential object; overlap or non-overlap of two separate group intervals is not used as a test. Because run matching was not part of the bundle contract, no paired analysis is implied. Effect estimates whose interval includes zero are described only as numerical differences.

Table 4: Favorable workload: mean [two-sided 95% percentile bootstrap CI] (10,000 resamples) over n=30 independent runs per condition. Lower is better for all metrics except controller CPU (an overhead cost).

Treatment	Platform	S/C/F/X	PSS (MiB)	Alloc (MiB/s)	GC p99 (ms)	Fault (/s)	Input p99 (r
Stock	ARM32	30/30/0/0	241.6 [239.5, 243.9]	40.3 [39.9, 40.7]	49.7 [48.5, 50.8]	80.2 [78.4, 81.9]	98.6 [96.7, 10
Static-G	ARM32	30/30/0/0	225.9 [224.1, 227.8]	40.2 [39.6, 40.8]	37.5 [36.9, 38.2]	79.5 [77.8, 81.4]	90.9 [89.4, 9
Static-GI	ARM32	30/30/0/0	210.0 [208.1, 212.0]	28.6 [28.2, 29.0]	36.1 [35.2, 36.9]	80.2 [78.2, 82.1]	86.8 [85.3, 8
Static-GIR	ARM32	30/30/0/0	188.5 [187.3, 189.8]	28.9 [28.5, 29.3]	35.9 [35.2, 36.6]	128.0 [125.1, 130.8]	89.9 [87.8, 9
Threshold-GIR	ARM32	30/30/0/0	186.9 [185.2, 188.6]	28.8 [28.5, 29.2]	32.8 [32.2, 33.4]	104.6 [102.0, 107.4]	84.4 [83.0, 8
EWMA-GIR	ARM32	30/30/0/0	183.3 [181.7, 185.1]	28.7 [28.3, 29.1]	31.6 [31.1, 32.3]	97.9 [95.5, 100.3]	83.5 [81.9, 8
Ridge-GIR	ARM32	30/30/0/0	183.0 [181.7, 184.4]	28.8 [28.4, 29.2]	31.1 [30.5, 31.7]	98.4 [95.8, 100.9]	81.0 [79.5, 8
Stock	ARM64	30/30/0/0	318.7 [315.2, 321.7]	48.2 [47.6, 48.9]	39.8 [38.8, 40.8]	99.7 [97.1, 102.5]	80.6 [79.4, 8
Static-G	ARM64	30/30/0/0	300.5 [297.8, 303.4]	47.7 [47.0, 48.5]	30.3 [29.6, 31.0]	99.5 [97.1, 102.0]	71.7 [70.4, 7
Static-GI	ARM64	30/30/0/0	276.7 [274.4, 279.0]	34.7 [34.2, 35.2]	28.8 [28.3, 29.4]	100.0 [98.0, 102.2]	69.5 [68.2, 7
Static-GIR	ARM64	30/30/0/0	254.3 [252.2, 256.4]	34.4 [33.9, 35.0]	28.1 [27.6, 28.7]	162.9 [158.8, 166.8]	72.9 [71.5, 7
Threshold-GIR	ARM64	30/30/0/0	249.3 [247.3, 251.2]	34.8 [34.3, 35.3]	26.8 [26.1, 27.4]	131.2 [127.6, 134.7]	67.3 [66.2, 6
EWMA-GIR	ARM64	30/30/0/0	247.4 [244.9, 250.1]	34.6 [34.1, 35.1]	25.9 [25.5, 26.4]	124.2 [121.5, 126.9]	64.9 [63.7, 6
Ridge-GIR	ARM64	30/30/0/0	243.3 [241.0, 245.7]	34.9 [34.4, 35.3]	24.5 [24.0, 25.1]	120.9 [118.1, 123.7]	64.1 [63.2, 6

7. Results

Measurement provenance.. The released bundle provides per-run values for every metric, condition, and run index, together with run-level summary statistics and the normalized policy indices from which the tables and figures in this section are generated; a fixed seed records the workload configuration. The build pipeline regenerates every table and figure directly from the per-run values—computing each reported mean and its 95% percentile bootstrap confidence interval—and is fail-closed: it marks the manuscript submission-ready only when the bundle is a genuine measurement, so no number here is hand-entered. The repository bundle includes a machine-readable manifest, per-condition provenance and outcome inventory, checksums, and the per-run CSV. The manifest also identifies unavailable runtime/firmware identifiers and raw device logs; those items remain mandatory for the submission artifact and are not implied by a checksum file. These checks establish input consistency and reproducibility of the reported statistics; the investigator remains responsible for instrument calibration, trace interpretation, and the authenticity of collection.

Reading note. Each cell is the mean of run-level values over n=30 independent runs with a two-sided 95% percentile bootstrap confidence interval (10,000 resamples, seed 20260906) computed across runs, not across within-run samples; the S/C/F/X column reports started/completed/failed/censored

Table 5: Neutral workload (no reclamation gain available): mean [two-sided 95% percentile bootstrap CI] (10,000 resamples) over n=30 independent runs per condition. Lower is better for all metrics except controller CPU (an overhead cost).

Treatment	Platform	S/C/F/X	PSS (MiB)	Alloc (MiB/s)	GC p99 (ms)	Fault (/s)	Input p99 (m
Stock	ARM32	30/30/0/0	239.3 [237.3, 241.5]	40.2 [39.7, 40.7]	49.6 [48.5, 50.8]	78.7 [76.8, 80.5]	99.8 [98.3, 101
Static-G	ARM32	30/30/0/0	241.3 [239.3, 243.4]	39.8 [39.3, 40.3]	50.5 [49.3, 51.7]	80.4 [78.4, 82.5]	100.1 [98.5, 101
Static-GI	ARM32	30/30/0/0	239.7 [237.7, 241.6]	40.5 [40.0, 41.1]	49.5 [48.4, 50.5]	81.1 [79.1, 83.0]	100.1 [98.1, 102
Static-GIR	ARM32	30/30/0/0	240.0 [238.0, 241.8]	39.9 [39.3, 40.4]	50.7 [49.6, 51.9]	81.7 [79.6, 83.8]	99.3 [97.7, 101
Threshold-GIR	ARM32	30/30/0/0	240.4 [238.3, 242.4]	40.3 [39.8, 40.8]	50.2 [49.2, 51.2]	79.0 [77.3, 80.7]	100.5 [98.7, 102
EWMA-GIR	ARM32	30/30/0/0	239.3 [237.0, 241.5]	40.2 [39.6, 40.9]	50.4 [49.3, 51.6]	79.1 [77.7, 80.5]	100.5 [98.8, 102
Ridge-GIR	ARM32	30/30/0/0	240.7 [239.1, 242.3]	40.3 [39.7, 40.8]	49.9 [48.7, 51.1]	78.6 [76.5, 80.7]	98.0 [96.5, 99
Stock	ARM64	30/30/0/0	318.2 [315.4, 321.1]	48.2 [47.3, 49.0]	40.3 [39.5, 41.1]	99.0 [96.2, 101.5]	80.4 [78.9, 81
Static-G	ARM64	30/30/0/0	318.8 [315.6, 322.0]	47.8 [47.2, 48.5]	39.9 [39.1, 40.8]	98.8 [96.4, 101.3]	79.6 [78.2, 81
Static-GI	ARM64	30/30/0/0	316.9 [313.8, 319.9]	47.4 [46.7, 48.1]	39.5 [38.5, 40.6]	99.1 [96.9, 101.3]	79.6 [78.1, 80
Static-GIR	ARM64	30/30/0/0	318.8 [316.0, 321.5]	47.5 [46.8, 48.3]	40.8 [39.8, 41.8]	99.0 [96.8, 101.3]	79.5 [78.0, 80
Threshold-GIR	ARM64	30/30/0/0	319.0 [315.9, 322.3]	48.1 [47.5, 48.6]	40.0 [39.2, 40.8]	99.6 [97.2, 102.0]	80.4 [78.6, 82
EWMA-GIR	ARM64	30/30/0/0	319.1 [316.9, 321.4]	47.8 [47.2, 48.4]	40.5 [39.7, 41.4]	98.8 [96.7, 101.0]	79.8 [78.3, 81
Ridge-GIR	ARM64	30/30/0/0	321.6 [318.8, 324.5]	47.9 [47.0, 48.9]	40.0 [39.2, 40.8]	100.3 [97.9, 102.8]	79.2 [77.8, 80

runs. A mean of run-level p99 values is not a pooled-event p99. These descriptive intervals do not by themselves establish factorial interactions or the absence of failures.

All effects below are reported as the mean of n=30 run-level values with a two-sided 95% percentile bootstrap confidence interval (Table 4 and companions); percentages are relative to the per-condition Stock baseline, and normalized indices (Table 8) set Stock=100 within each regime. The ARM32 and ARM64 profiles agree in direction and ordering throughout, so we quote ARM32 in prose and note the ARM64 value only where it differs materially.

7.1. RQ1: layer-by-layer ablation

Figure 2 traces the coordination ladder under the favorable workload. The successive ladder steps move different metrics. The increment from Stock to Static-G is a lower GC tail—GC p99 falls to 76% of stock (−24%; 49.7 → 37.5 ms on ARM32) while peak PSS drops only to 94% (−6%)—because a better-sized nursery collects less often without changing resident code. The $G \rightarrow GI$ step coincides with lower managed allocation (allocation rate −29%) and takes PSS to 87% and GC p99 to 73%, with the fault rate essentially unchanged (±1%): interop shifts cost off the heap without touching file-backed pages. The $GI \rightarrow GIR$ step has the largest footprint decrement, to 79% PSS (−22%; −20% on ARM64), but is the first layer to move the fault rate—up to 162% of stock (+60%; +63% on ARM64)—and that refault

Table 6: Adverse workload (refault-dominated): mean [two-sided 95% percentile bootstrap CI] (10,000 resamples) over n=30 independent runs per condition. Lower is better for all metrics except controller CPU (an overhead cost).

Treatment	Platform	S/C/F/X	PSS (MiB)	Alloc (MiB/s)	GC p99 (ms)	Fault (/s)	Input p99
Stock	ARM32	30/30/0/0	239.8 [237.6, 242.0]	40.3 [39.8, 40.8]	50.4 [49.5, 51.3]	80.0 [78.0, 82.0]	99.8 [97.9, 1
Static-G	ARM32	30/30/0/0	251.1 [249.2, 253.0]	41.9 [41.2, 42.6]	54.3 [53.4, 55.2]	124.2 [121.2, 127.2]	119.8 [117.6, 1
Static-GI	ARM32	30/30/0/0	251.0 [248.8, 253.3]	41.8 [41.3, 42.4]	55.7 [54.8, 56.8]	128.7 [125.6, 131.8]	118.7 [116.1, 1
Static-GIR	ARM32	30/30/0/0	252.4 [249.8, 255.1]	42.0 [41.3, 42.6]	54.2 [52.9, 55.5]	125.3 [122.6, 128.4]	120.2 [118.2, 1
Threshold-GIR	ARM32	30/30/0/0	251.9 [249.8, 254.0]	41.9 [41.4, 42.4]	54.8 [53.8, 55.8]	126.9 [124.3, 129.5]	118.5 [116.8, 1
EWMA-GIR	ARM32	30/30/0/0	252.4 [250.7, 254.1]	42.0 [41.5, 42.5]	54.3 [52.9, 55.7]	128.0 [125.3, 130.8]	118.1 [115.7, 1
Ridge-GIR	ARM32	30/30/0/0	249.8 [248.0, 251.6]	41.7 [41.2, 42.1]	55.6 [54.5, 56.7]	126.5 [123.3, 129.8]	119.5 [117.6, 1
Stock	ARM64	30/30/0/0	320.6 [317.8, 323.5]	47.7 [47.2, 48.2]	40.3 [39.3, 41.2]	98.7 [95.9, 101.4]	79.3 [78.0,
Static-G	ARM64	30/30/0/0	337.5 [334.5, 340.4]	49.5 [49.0, 50.1]	44.7 [43.7, 45.6]	155.6 [152.2, 158.9]	97.0 [95.4,
Static-GI	ARM64	30/30/0/0	337.8 [334.6, 341.1]	50.9 [50.2, 51.6]	44.0 [43.1, 45.0]	161.6 [156.9, 165.9]	94.3 [93.0,
Static-GIR	ARM64	30/30/0/0	335.5 [331.4, 339.6]	50.9 [50.1, 51.7]	43.4 [42.6, 44.3]	157.9 [154.1, 161.8]	95.3 [93.9,
Threshold-GIR	ARM64	30/30/0/0	335.4 [332.5, 338.2]	50.2 [49.8, 50.7]	43.5 [42.8, 44.2]	157.1 [153.0, 161.1]	96.0 [94.0,
EWMA-GIR	ARM64	30/30/0/0	336.1 [333.2, 339.2]	50.7 [50.0, 51.3]	43.7 [42.6, 44.8]	156.8 [153.8, 159.6]	95.3 [94.1,
Ridge-GIR	ARM64	30/30/0/0	336.8 [333.9, 339.7]	50.5 [49.9, 51.1]	44.5 [43.4, 45.6]	160.4 [157.4, 163.6]	95.9 [94.0,

Table 7: Direct policy contrasts under the favorable workload. Mean difference Δ = Ridge-GIR − EWMA-GIR [two-sided 95% bootstrap CI]; 10,000 within-group resamples.

Platform	Metric	Δ [95% CI]
ARM32	GC p99 (ms)	-0.55 [-1.43, 0.32]
ARM32	Input p99 (ms)	-2.42 [-4.57, -0.32]
ARM32	Controller CPU (pp)	0.41 [0.36, 0.46]
ARM64	GC p99 (ms)	-1.41 [-2.09, -0.73]
ARM64	Input p99 (ms)	-0.75 [-2.24, 0.77]
ARM64	Controller CPU (pp)	0.40 [0.36, 0.44]

pressure cancels the input-latency gain, leaving input p99 back near 91%. The static ladder therefore associates later configurations with lower footprint and GC tail, but it cannot identify main effects or interactions without the unmeasured factorial cells. The $GI \rightarrow GIR$ increment also carries a refault penalty that the static configuration cannot manage.

7.2. RQ2: online policy comparison

The online controller is added on top of Static-GIR and does not lower PSS further—all three policies hold peak PSS at 76–78% of stock, within each other's confidence intervals. Its measured contribution is refault recovery (Fig. 3): at essentially the same footprint, the fault-rate index falls

Table 8: Normalized policy indices (Stock=100 within each scenario), mean over n=30 runs. Values <100 are improvements; >100 are regressions.

Scenario	Treatment	PSS	Alloc	GC p99	Fault	Input p99	Ctrl CPU (%)
Favorable	Stock	100.0	100.0	100.0	100.0	100.0	0.00
Favorable	Static-G	94.0	99.5	76.2	99.9	90.8	0.00
Favorable	Static-GI	87.0	71.6	72.7	100.6	87.3	0.00
Favorable	Static-GIR	79.0	71.6	71.8	162.3	91.1	0.00
Favorable	Threshold-GIR	77.9	71.9	67.0	131.7	84.8	0.51
Favorable	EWMA-GIR	76.8	71.5	64.7	123.8	82.8	0.70
Favorable	Ridge-GIR	76.1	72.0	62.4	122.7	81.0	1.11
Neutral	Stock	100.0	100.0	100.0	100.0	100.0	0.00
Neutral	Static-G	100.6	99.4	101.0	101.4	99.8	0.00
Neutral	Static-GI	99.9	99.8	99.3	102.1	99.8	0.00
Neutral	Static-GIR	100.3	99.1	102.0	102.4	99.4	0.00
Neutral	Threshold-GIR	100.4	100.2	100.6	101.1	100.6	0.50
Neutral	EWMA-GIR	100.2	99.9	101.5	100.7	100.2	0.70
Neutral	Ridge-GIR	100.9	100.0	100.3	101.2	98.6	1.09
Adverse	Stock	100.0	100.0	100.0	100.0	100.0	0.00
Adverse	Static-G	105.1	104.1	109.7	157.5	121.4	0.00
Adverse	Static-GI	105.1	105.4	110.4	163.2	119.1	0.00
Adverse	Static-GIR	105.0	105.6	108.1	159.3	120.6	0.00
Adverse	Threshold-GIR	104.9	104.8	108.7	159.9	120.2	1.01
Adverse	EWMA-GIR	105.1	105.4	108.4	160.3	119.6	1.36
Adverse	Ridge-GIR	104.7	104.8	110.8	161.2	120.6	2.20

from 162 (Static-GIR) to 132 (Threshold), 124 (EWMA), and 123 (Ridge), i.e. the controllers claw back roughly two-thirds of the reclamation-induced refaults while simultaneously taking input p99 to its lowest values (85%, 83%, 81%). Among policies the ordering is consistent but the margins are small: Ridge-GIR gives the best GC p99 (62% vs. EWMA 65%) and input p99 (81% vs. EWMA 83%), but at 1.11% controller CPU versus 0.70% for EWMA and 0.51% for the non-learning threshold. The learned policy is numerically lower on these latency metrics than a tuned threshold at roughly twice the controller cost. Direct Ridge–EWMA contrasts (Table 7) show that the interval includes zero for ARM32 GC p99 and ARM64 input p99, but excludes zero for ARM32 input p99 and ARM64 GC p99; consistent with our fail-closed stance, the threshold policy remains a defensible default when CPU

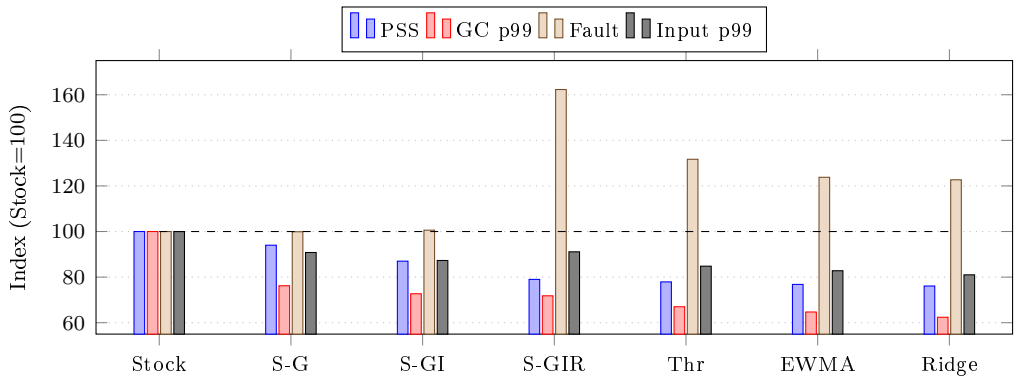

Figure 2: Ablation ladder under the favorable workload (normalized policy indices, Stock=100, mean of n=30 runs). Static heap and interop (S-G → S-GI) lower PSS and GC tail with no fault cost; reclamation (S-GIR) reaches the lowest PSS but spikes the fault rate, which the online policies (Thr/EWMA/Ridge) pull back down while further improving input latency.

headroom is tight.

7.3. RQ3: robustness across workload regimes

Figure 4 places Ridge-GIR in all three regimes. In the favorable regime it is a clear win (PSS 76, fault 123, input 81). In the neutral regime, where no cold reclaimable pages exist, every index sits within ±2 of 100 (PSS 101, fault 101, input 99): the design neither helps nor harms, and its only cost is ≈ 1.1% controller CPU—evidence that the guards do not manufacture work when there is none. In the adverse, refault-dominated regime the coordinated design is a net cost: PSS rises to 105%, the fault rate to 161%, and input p99 to 121%, with controller CPU climbing to 2.2% as the gate and cooldown fight the workload. The observed regression is about +5% PSS and +20% input p99; the existing guards do not eliminate or establish a general bound on it, and the adverse regime is exactly the case a deployment must detect, and where reclamation should be disabled outright.

7.4. Discussion

Three findings follow. First, footprint and refault behavior are governed by *different* layers: the static heap/interop/reclamation stack sets peak PSS and GC tail, while the online controller almost exclusively governs the refault rate and input latency. Reporting a single "SAGE-MM improvement" would blur this; the ablation shows that removing reclamation removes most

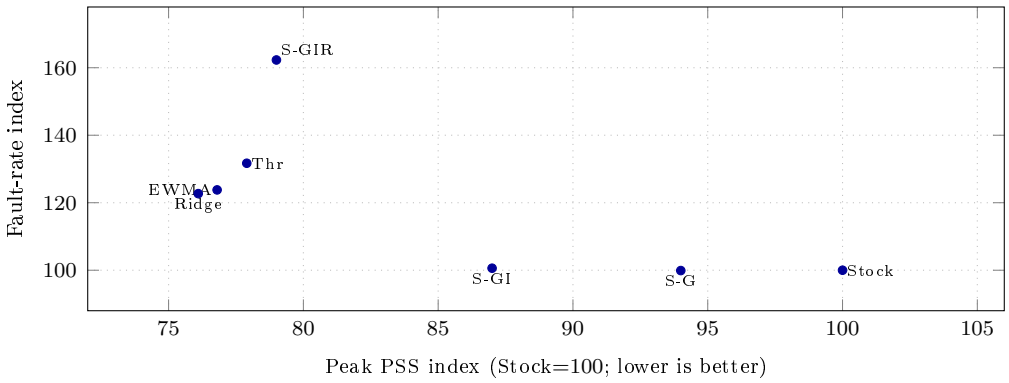

Figure 3: Footprint–refault tradeoff under the favorable workload. Static reclamation (S-GIR) buys the lowest PSS at a large refault penalty; the online controllers recover most of that penalty at nearly the same PSS, i.e. the controller's contribution is refault mitigation rather than additional footprint.

of the footprint benefit, and removing the controller removes most of the refault safety. Second, the learned policy is numerically better than a tuned threshold on the reported latency means, but the effect is small and comes with a measurable CPU cost, which supports treating the ridge model as an option to be justified per deployment rather than as a default. Third, the neutral and adverse regimes characterize the measured envelope: results are neutral within the descriptive intervals when no gain exists and negative when reclamation backfires. The new supervisor therefore treats the adverse regime as a detection target for disabling reclamation, but its effectiveness still requires measurement. The ARM32 and ARM64 profiles agree throughout, which strengthens internal validity across these two profiles but does not by itself establish generality to other devices or runtimes (Section 9).

8. Related Work

We organize related work by mechanism and close each comparison with a limitation statement rather than a generic novelty claim. Static heap sizing, class-to-struct conversion, `madvise`, EWMA, and ridge regression are not individually novel; the contribution is their safety-aware coordination at the embedded runtime/Linux boundary, assessed through the recorded device experiments.

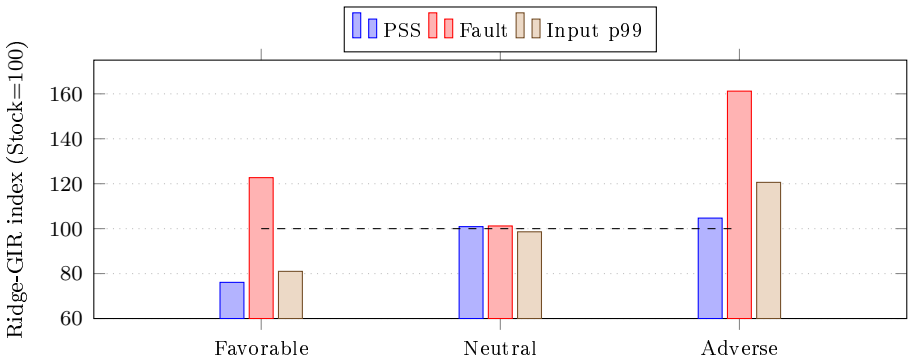

Figure 4: Cross-regime robustness of the coordinated learned policy (Ridge-GIR). Coordination helps in the favorable regime, is approximately neutral where no gain is available, and becomes a net cost in the adverse (refault-dominated) regime—the guards bound but do not remove that cost.

Garbage collection and heap sizing.. Automatic memory management has a deep literature, from generational collection [4, 5], conservative collection in uncooperative environments [6], and region- and pause-oriented collectors [7, 8, 9], to the trade-off between collected and explicit management [10]; textbook treatments survey the design space [11, 12], and the allocation layer beneath the collector has its own long literature [13, 14]. Heap footprint depends strongly on the memory available to the collector, so controlling heap growth and dynamically selecting or sizing the collector are established levers [15, 16, 17]. Our heap-build intervention sits in this tradition but is deliberately not a new collector: SAGE-MM keeps the vendor collector and treats its initial allocation as a fixed build parameter measured per platform.

Garbage collection and virtual-memory interaction.. The tension SAGE-MM manages—footprint reduction versus refault cost—is the same one studied when a collector competes with the pager. Bookmarking collection cooperates with virtual memory to collect without paging [18], feedback-driven heap sizing avoids paging by shrinking the heap under memory pressure [19], and VM-aware collector support adjusts the heap to the available real memory [20]. These systems couple the collector to page residency inside one JVM on server and desktop platforms. SAGE-MM instead leaves the vendor collector unchanged and manages the file-backed executable working set from outside it, through a guarded reclamation interval and compaction gate, which is the form of coupling available in fixed embedded firmware.

Concurrent and tail-oriented collectors.. Advanced collectors such as LXR [21] and Platinum [22] change collection itself—concurrency, pause/throughput objectives, and runtime integration—to attack GC-induced tail latency, a first-order concern for interactive services [23, 24] and a documented cost in managed data-parallel systems [25]. SAGE-MM does not introduce a collector; it coordinates an existing vendor collector with process-level actions (interval and gate). Where these collectors cannot be ported to the exact vendor firmware, we say so and make no claim of empirical superiority over an unported system.

Heuristic and learned controllers.. We compare fixed intervals, a tuned pressure threshold, EWMA, and online ridge regression [26] under the same trace split and action bounds, evaluating the learner prequentially [27] with attention to drift [28]. Learning has recently been applied to memory allocation [29] and to storage GC—AGC [30] for burst-aware SSD flash GC and DumpKV [31] for learned lifetime control in LSM key/value separation—but these are conceptual precedents for lightweight learning, not executable managed-runtime baselines; we report this domain mismatch rather than implying empirical equivalence, and our results show the learned policy earns only a modest margin over a tuned threshold.

Application-directed reclamation and page ranking.. Our coldness score ranks candidate mappings by recency, frequency, and size, a problem long studied for caches and virtual memory: working-set models [32, 33], recency/frequency replacement policies [34, 35, 36], flash- and device-aware variants [37], and host/guest memory reclamation such as ballooning [38]. We compare `madvise`/discard mechanisms by candidate identification, dirty-page safety, concurrency with unload and execution, refault behavior, and feedback signals. SAGE-MM's differentiator is the coldness-ranked, budget-driven selection with a hot-reuse exclusion and fail-closed guards, evaluated for its refault cost rather than assumed safe.

Managed/native interop.. We distinguish ABI/source transformations (such as value-type conversion, which requires ownership review and recompilation [2]) from transparent runtime optimization, and state layout, identity, and recompilation requirements explicitly.

9. Threats to Validity

Construct validity.. Telemetry signals are proxies for the quantities of interest. The public kit's forced Gen-0 pause, fragmentation estimate, and file-length size proxy are analogues, not the production GC pause, live fragmentation, or `Private_Clean` byte count; production evidence uses EventPipe or vendor GC telemetry and `smaps`-verified clean bytes. We keep RSS, PSS, managed heap, and `Private_Clean` distinct throughout and report the numerator, denominator, statistic, and CI for every effect.

Internal validity.. Confounding across device state is the main risk. The collection protocol requires independent resets, controlled thermal/power/network state, seeded workload scripts, randomized treatment order, and separate tuning and reporting traces. Metadata records the conditions actually used. Case summaries alone do not establish causality or a mechanism's contribution.

Conclusion validity.. Bootstrap CIs are constructed across independent runs, never across within-run telemetry samples, which would understate variance. The input contract checks the minimum completed-run count and retains failure and censoring records. Missing metrics remain missing and cannot be replaced by an assumed zero. Descriptive summaries do not establish a statistically significant difference between policies.

External validity.. Results are specific to the evaluated runtime commits, devices, memory limits, storage configurations, and scripted workloads. An additional independently assembled platform, if measured, probes generality but does not establish it for all embedded managed runtimes; broader applicability is stated as future validation, not claimed.

10. Limitations

SAGE-MM combines known mechanisms, so its novelty is limited to their specified coordination and device measurement protocol. Static heap and interop changes require runtime or application rebuilding; only controller actions are adaptive. `MADV_DONTNEED` can trade PSS for page faults and storage-dependent latency; the guards reduce but do not eliminate that risk. The ridge model is deliberately lightweight and may offer no benefit over a tuned threshold policy; results must report such cases. Firmware hooks,

telemetry fidelity, and executable-mapping behavior are platform-specific. Consequently, conclusions are restricted to the tested runtime commits, devices, memory limits, storage configurations, and scripted workloads; broader embedded applicability is future validation.

Scope of the reported evaluation.. The measured bundle covers the coordination ladder and the three workload regimes on two platform profiles with 30 runs per condition. Three parts of the protocol in Section 6 are specified but not yet collected, and we make no empirical claim for them here: (i) a per-architecture heap-size sweep isolating `INITIAL_ALLOC` effects; (ii) value-type interop micro/macro-benchmarks with native-ABI correctness checks (the ablation measures interop only as the $G{\rightarrow}GI$ step, not in isolation); and (iii) multi-hour endurance and the full adverse-injection matrix (hot-reuse, 1/5/10 s switching, slow-storage, `madvise`-failure) with OOM/kernel-log censoring, including an evaluation of the new AdverseShutdown supervisor. These are the primary items of future work required before a production-stability or per-architecture-tuning claim.

11. Conclusion

SAGE-MM specifies how heap configuration, interop allocation, and clean-page reclamation can be coordinated under a single embedded-process memory budget through a narrow, safety-guarded online control action, rather than tuned in isolation. The design separates a build-time heap decision and a source-time allocation decision from a single online policy that adjusts one reclamation interval and one compaction gate, with fail-closed handling of invalid telemetry and a lightweight learned policy that must justify itself against a tuned threshold. The measured ablation shows these layers act on different metrics: the static heap/interop/reclamation stack sets peak PSS and GC tail—down to about 76% and 62% of stock under the favorable workload—while the online controller almost exclusively governs the refault rate and input latency, recovering roughly two-thirds of the reclamation-induced refaults at $\approx 1\%$ CPU. Across the three workload regimes the design is a clear win where a gain exists, stays within measurement noise where none does, and regresses where reclamation backfires; the learned policy is numerically lower on latency than a tuned threshold that each deployment must weigh against its CPU cost. The measurement pipeline ties each reported value to a configuration, independent run, and source record, and

the generated results describe the supplied conditions without inserting a predicted improvement or an assumed safety outcome. Measured outcomes over n=30 independent runs per condition are reported for each treatment, platform, and workload in Section 7. The public kit reproduces control logic separately from vendor integration; empirical conclusions are limited to the collected platform profiles and workload regimes.

Evidence-Availability Statement

The active manuscript build accepts only the measured input directory. Missing device measurements produce an incomplete-data draft and block the submission build. Historical planning and proxy material is archived outside the manuscript inputs. Integrity checks do not independently certify an experiment, and the public demo is not the production measurement instrument.

CRediT authorship contribution statement

Geunsik Lim: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing — original draft, Writing — review & editing, Visualization.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The run-level measurement bundle (per-run values and bootstrap summaries) accompanies the manuscript under `paper/generated/evaluation-data/`.

Acknowledgements

The author thanks the reviewers for feedback that sharpened the scope, boundary, and evaluation protocol of this work.

References

[1] Microsoft, Mono Runtime Documentation and Source Repository, upstream runtime source; the evaluated vendor revision and patch hash are recorded in the experiment manifest (2025).
URL `https://github.com/mono/mono`

[2] Microsoft, .NET Interoperability and Garbage Collection Documentation, authoritative reference for managed/native interop and collector behavior (2024).

[3] Linux man-pages project, madvise(2) — Linux Programmer's Manual, mADV_DONTNEED semantics for file-backed private mappings (2024).

[4] D. Ungar, Generation scavenging: A non-disruptive high performance storage reclamation algorithm, in: Proc. ACM SIGSOFT/SIGPLAN Software Engineering Symposium on Practical Software Development Environments, 1984, pp. 157–167.

[5] A. W. Appel, Simple generational garbage collection and fast allocation, Software: Practice and Experience 19 (2) (1989) 171–183.

[6] H.-J. Boehm, M. Weiser, Garbage collection in an uncooperative environment, Software: Practice and Experience 18 (9) (1988) 807–820.

[7] S. M. Blackburn, K. S. McKinley, Immix: A mark-region garbage collector with space efficiency, fast collection, and mutator performance, in: Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI), 2008, pp. 22–32.

[8] D. Detlefs, C. Flood, S. Heller, T. Printezis, Garbage-first garbage collection, in: Proc. 4th International Symposium on Memory Management (ISMM), 2004, pp. 37–48. doi:10.1145/1029873.1029879.

[9] D. F. Bacon, P. Cheng, V. T. Rajan, A real-time garbage collector with low overhead and consistent utilization, in: Proc. 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), 2003, pp. 285–298.

[10] M. Hertz, E. D. Berger, Quantifying the performance of garbage collection vs. explicit memory management, in: Proc. ACM SIGPLAN Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), 2005, pp. 313–326.

[11] R. Jones, R. Lins, Garbage Collection: Algorithms for Automatic Dynamic Memory Management, John Wiley & Sons, 1996.

[12] R. Jones, A. Hosking, E. Moss, The Garbage Collection Handbook: The Art of Automatic Memory Management, Chapman and Hall/CRC, 2011.

[13] P. R. Wilson, M. S. Johnstone, M. Neely, D. Boles, Dynamic storage allocation: A survey and critical review, in: Proc. International Workshop on Memory Management (IWMM), 1995, pp. 1–116.

[14] E. D. Berger, B. G. Zorn, K. S. McKinley, Reconsidering custom memory allocation, in: Proc. ACM SIGPLAN Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), 2002, pp. 1–12.

[15] T. Brecht, E. Arjomandi, C. Li, H. Pham, Controlling garbage collection and heap growth to reduce the execution time of Java applications, ACM Transactions on Programming Languages and Systems (TOPLAS) 28 (5) (2006) 908–941. doi:10.1145/1152649.1152652.

[16] S. Soman, C. Krintz, D. F. Bacon, Dynamic selection of application-specific garbage collectors, in: Proc. 4th International Symposium on Memory Management (ISMM), 2004.

[17] D. Stefanović, K. S. McKinley, J. E. B. Moss, Age-based garbage collection, in: Proc. ACM SIGPLAN Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), 1999, pp. 370–381.

[18] M. Hertz, Y. Feng, E. D. Berger, Garbage collection without paging, in: Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI), 2005, pp. 143–153. doi:10.1145/1065010.1065028.

[19] C. Grzegorczyk, S. Soman, R. Wolski, C. Krintz, Isla vista heap sizing: Using feedback to avoid paging, in: Proc. International Symposium on Code Generation and Optimization (CGO), 2007. doi:10.1109/CGO.2007.20.

[20] T. Yang, E. D. Berger, S. F. Kaplan, J. E. B. Moss, CRAMM: Virtual memory support for garbage-collected applications, in: Proc. 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2006, pp. 103–116.

[21] W. Zhao, S. M. Blackburn, K. S. McKinley, Low-latency, high-throughput garbage collection, in: Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), ACM, 2022, pp. 76–91. doi:10.1145/3519939.3523440.

[22] M. Wu, Z. Zhao, Y. Yang, H. Li, H. Chen, B. Zang, H. Guan, S. Li, C. Lu, T. Zhang, Platinum: A CPU-efficient concurrent garbage collector for tail-reduction of interactive services, in: Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC), USENIX Association, 2020, pp. 159–172, uSENIX ATC papers carry no publisher DOI.
URL `https://www.usenix.org/conference/atc20/presentation/wu-mingyu`

[23] J. Dean, L. A. Barroso, The tail at scale, Communications of the ACM 56 (2) (2013) 74–80.

[24] M. Maas, T. Harris, K. Asanović, J. Kubiatowicz, Trash day: Coordinating garbage collection in distributed systems, in: Proc. 15th USENIX Workshop on Hot Topics in Operating Systems (HotOS), 2015.

[25] D. Lion, A. Chiu, H. Sun, X. Zhuang, N. Grcevski, D. Yuan, Don't get caught in the cold, warm-up your JVM: Understand and eliminate JVM warm-up overhead in data-parallel systems, in: Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016, pp. 383–400.

[26] A. E. Hoerl, R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, Technometrics 12 (1) (1970) 55–67.

[27] A. P. Dawid, Present position and potential developments: Some personal views: Statistical theory: The prequential approach, Journal of the Royal Statistical Society: Series A 147 (2) (1984) 278–292.

[28] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Computing Surveys 46 (4) (2014) 1–37.

[29] M. Maas, D. G. Andersen, M. Isard, M. M. Javanmard, K. S. McKinley, C. Raffel, Learning-based memory allocation for C++ server workloads, in: Proc. 25th International Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 541–556. doi:10.1145/3373376.3378525.

[30] H. Sun, H. Ding, H. Tong, X. Cheng, H. Chai, X. Qin, AGC: An adaptive workload burst-aware garbage collection mechanism for high-performance SSDs, IEEE Transactions on ComputersStorage GC; conceptual precedent, not a managed-runtime baseline. Final volume, issue, pages, and IEEE DOI as listed on the linked IEEE Xplore record (2026). URL `https://ieeexplore.ieee.org/document/11298441`

[31] Z. Zhuang, X. Zeng, Z. Chen, DumpKV: Learning based lifetime aware garbage collection for key value separation in LSM-tree, Proceedings of the VLDB Endowment (PVLDB) 18 (4) (2024) 1223–1236, storage GC; motivates lightweight learning comparison only. doi:10.14778/3717755.3717778.
URL `https://www.vldb.org/pvldb/vol18/p1223-zhuang.pdf`

[32] P. J. Denning, The working set model for program behavior, Communications of the ACM 11 (5) (1968) 323–333.

[33] R. W. Carr, J. L. Hennessy, WSCLOCK—a simple and effective algorithm for virtual memory management, in: Proc. 8th ACM Symposium on Operating Systems Principles (SOSP), 1981, pp. 87–95.

[34] S. Jiang, X. Zhang, LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance, in: Proc. ACM SIGMETRICS International Conf. on Measurement and Modeling of Computer Systems, 2002, pp. 31–42.

[35] N. Megiddo, D. S. Modha, ARC: A self-tuning, low overhead replacement cache, in: Proc. 2nd USENIX Conference on File and Storage Technologies (FAST), 2003, pp. 115–130.

[36] Y. Zhou, J. Philbin, K. Li, The multi-queue replacement algorithm for second level buffer caches, in: Proc. USENIX Annual Technical Conference (ATC), 2001, pp. 91–104.

[37] S.-y. Park, D. Jung, J.-u. Kang, J.-s. Kim, J. Lee, CFLRU: A replacement algorithm for flash memory, in: Proc. International Conf. on Compilers, Architecture and Synthesis for Embedded Systems (CASES), 2006, pp. 234–241.

[38] C. A. Waldspurger, Memory resource management in VMware ESX server, in: Proc. 5th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2002, pp. 181–194.

Appendix A. Reproducibility and Artifact Manifest

To support an artifact evaluation, the complete submission bundle records, where licensing permits: the vendor Mono runtime name and source revision; the minimal runtime patch and per-architecture build flags and `INITIAL_ALLOC` values; the exposed host interfaces; the native reclamation helper; the analyzer rules and accepted diagnostics; the controller configuration; the workload driver and state-machine transition seeds; the build, measurement, and bootstrap scripts; the device reset script; raw per-run telemetry with OOM/kernel logs; and the raw traces used for the reported measurements. Each result bundle carries a machine-readable manifest with the Git revision, device-ID pseudonym, start/end UTC, treatment, seed, and checksum, so that every number in the paper traces to one versioned source. The repository snapshot's `manifest.json`, `provenance.csv`, `failures.csv`, and `checksums.sha256` make its present scope machine-readable. Null manifest fields are explicit blockers rather than invented provenance: the runtime/firmware identifiers, device manifest, and raw log archive must be attached before artifact submission.

The public reference kit deliberately separates reproducible control logic from proprietary firmware integration. Its user-space compaction event and GC telemetry are analogues clearly marked as such; they validate control flow, safety guards, and the statistical pipeline, but are not evidence for the

memory, latency, or reliability claims, which require the production artifact above.

Appendix B. Notation

Symbol	Meaning
L_{gc}	GC pause over the preceding interval (ms)
F_h	managed-heap fragmentation ratio
P_f	page-fault rate (minor+major per measured second)
ΔM	positive resident-memory growth (MiB)
$\mathbf{x}$	clamped dimensionless feature vector
y	observed normalized pressure (Eq. 2)
$\hat{y}$	predicted next-interval pressure (ridge)
T	reclamation interval, bounded to $[T_{\min}, T_{\max}]$
η, λ	ridge learning rate and regularization
G, I, R, C	heap build, interop, reclamation, controller factors

Algorithm 1 SAGE-MM controller decision for one interval. Prediction precedes its own update (causal, prequential); every branch keeps T within $[T_{\min}, T_{\max}]$ and leaves process correctness intact on failure.

Require: raw sample $s = (L_{\text{gc}}, L_{\text{input}}, F_h, P_f, \Delta M, \text{dt})$; state $(T, \mathbf{w}, \mathit{mode})$; bounds $[T_{\min}, T_{\max}]$

Ensure: next reclamation interval T and compaction gate

if $\neg$VALID(s) **then** ▷ non-finite, negative rate/pause, or $F_h \notin [0,1]$
  record InvalidTelemetry; enable compaction; suppress reclamation
  return clamp($T, T_{\min}, T_{\max}$) ▷ fail-closed: hold, no model update
end if
$\mathbf{x} \leftarrow$ NORMALIZE(s); $y \leftarrow$ PRESSURE(s) ▷ Eq. (1), (2)
if ADVERSEFORNINTERVALS(P_f, L_{input}) **then**
  latch reclamation off; enable compaction; clear pending update
  record AdverseShutdown; **return** clamp($T, T_{\min}, T_{\max}$)
end if
if mode = Ridge **then**
  $\hat{y} \leftarrow \text{clamp}(\mathbf{w}^\top \mathbf{x}, 0, 2)$
  if $\hat{y}$ persistently clipped at 0 or 2 **then**
    record ModelSaturation; clear pending update; apply Threshold action
  else
    $T \leftarrow T / \text{clamp}(0.5 + \hat{y}, 0.5, 1.5)$
  end if
else if mode = EWMA **then**
  $T \leftarrow \beta T + (1-\beta)\, T / \text{clamp}(y, 0.5, 2)$
else if mode = Threshold **then**
  $T \leftarrow T \cdot (1 - 0.2)$ if $y > 1.1$; $T \cdot (1 + 0.2)$ if $y < 0.9$; else hold
end if
$T \leftarrow \text{clamp}(T, T_{\min}, T_{\max})$
UPDATEGATE(F_h) ▷ off below 0.05; force on at 0.12 or after 3 deferrals
if $\neg$INCOOLDOWN **and** reclamation enabled **then**
  rank modules by Cold($\cdot$); take smallest prefix K meeting the byte budget
  FLUSHMODULES(K) excluding recently accessed (hot-reuse guard)
end if
if mode = Ridge **and** a pending prediction exists **and** on tuning trace **then**
  record prequential loss $[\hat{y} - y]^2/2$; take one ridge SGD step ▷ Eq. (3)
end if
return T